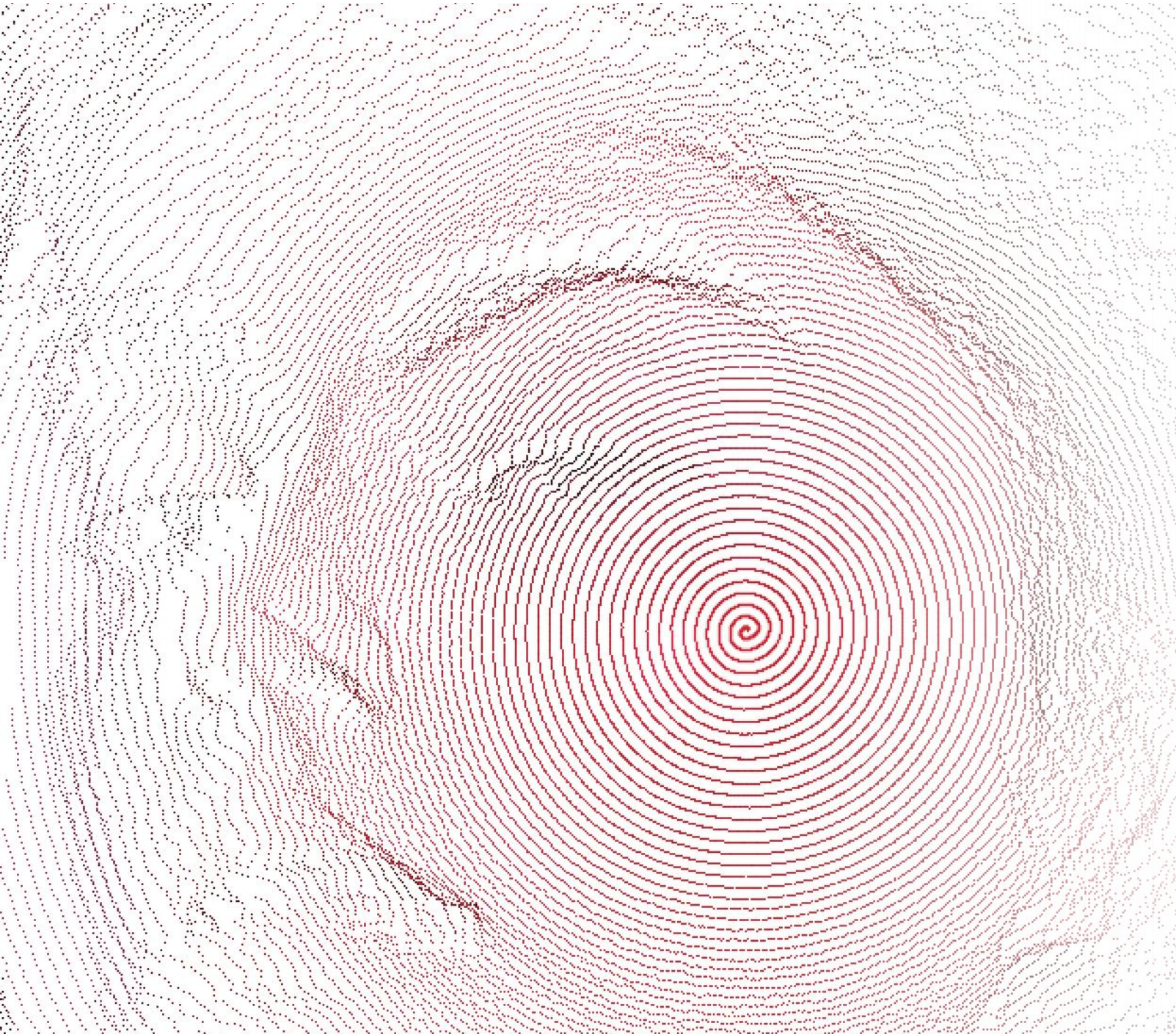
Portraiture
Design journal

Homework 4 — CS208

Yihong Lin



Concept

- Through the exhibition of works in class, my first inspiration is to show different colors of pictures with the thickness and density of lines seen in short videos. I want to make a portraiture by using parametric. Once I tried to use lines to show, it seemed a bit simple and boring. So I chose to use points and circles(2 versions switched by click the mouse). The color shade is represented by the crossing of an infinite number of concentric circles of equal width emitted by an infinite number of points.

Generative rules

1. Visual Object

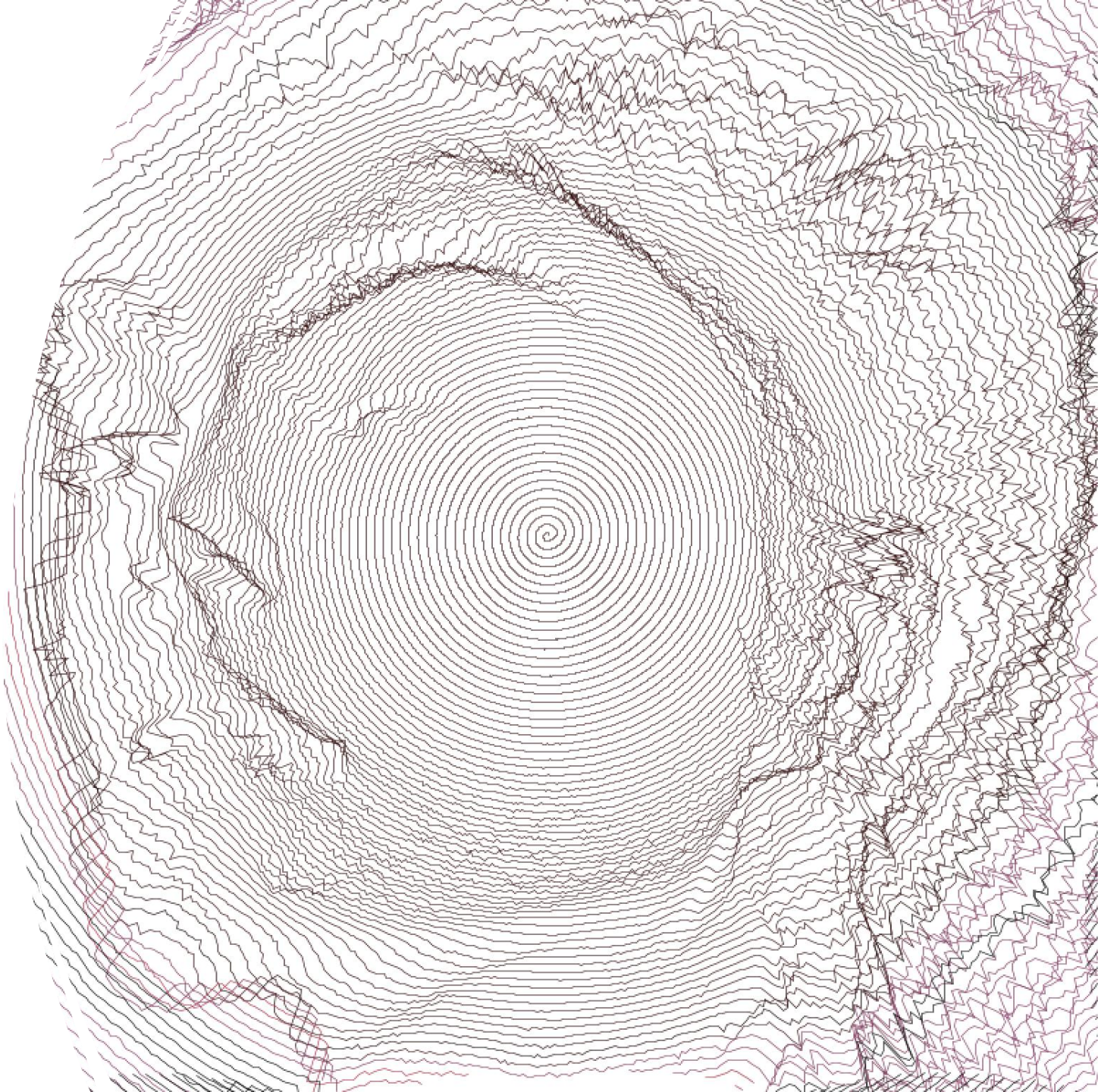
- A spiral-based pattern that is dynamically generated based on
- The color values of an image.
- The position of the mouse.
- A mathematical formula that defines the shape of the spiral.

2. Define the Parameters

- Image (img): The source image whose pixel colors influence the spiral's shape and color.
- Mouse Position (mouseX, mouseY): Controls the adj1 parameter, which affects the spiral's distortion.
- Time (frameCount): Used to create dynamic color effects.
- Spiral Shape Parameters:

  adj1: Controls the distortion of the spiral based on mouse position.

  adj2: Controls the amplitude of the sinusoidal distortion.



Generative rules

3. Define the Relationship and Scope of Each Parameter

Image and Color: The color of each point is derived from the pixel color of the image. The RGB values are dynamically mapped to create a color effect.

Mouse Position: The adj1 parameter is mapped to the sum of mouseX and mouseY, creating a direct relationship between mouse movement and spiral distortion.

Time: The frameCount is used to animate the color mapping, creating a time-based effect.

Spiral Shape: The shape is defined by a combination of:

A base spiral (t * cos(t), t * sin(t)).

A distortion term (pow(t, adj1) * sin(period * t)).

Rendering Mode: The boolean between drawing (point(x, y)) or (vertex(x, y)).

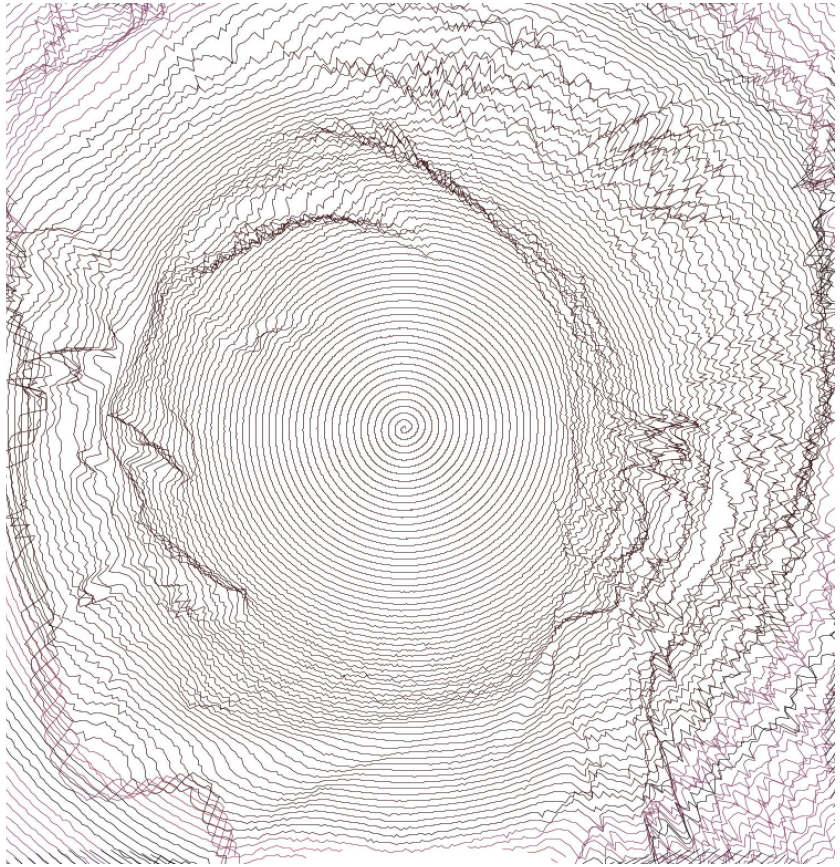
4. Define the Algorithms

Initialize Parameters: Load and resize the image. Set up the canvas and rendering parameters.

Generate the Spiral: Iterate over a range of values for t . For each value of t: Calculate the base spiral coordinates (x, y). Sample the color from the image at (x, y). Apply the distortion algorithm to calculate the final (x, y) position.

Render the Spiral: If form is true, draw the spiral as points. If form is false, draw the spiral as connected vertices.

Add Interactivity: Toggle the rendering mode (form) on mouse click.



Code

```
PImage img;
boolean form = true;

void setup() {
  size(800, 800);
  background(255);
  smooth(200);
  img = loadImage("image.jpg");
  img.resize(width, height);
}

void draw() {
  background(255);
  beginShape();
  noFill();
  stroke(0);
  float adj1 = map(mouseX + mouseY, 0, width + height, 0, 0.55);
  float adj2 = 0.02;
  float upperLimit = 100;
  float adjustedUpperLimit = 70;
  for (float t = 0; t <= upperLimit * TWO_PI; t += PI / 400) {
    float x, y, r, period;
    x = map(t * cos(t), -1 * adjustedUpperLimit * TWO_PI, adjustedUpperLimit * TWO_PI, 0, width);
    y = map(t * sin(t), -1 * adjustedUpperLimit * TWO_PI, adjustedUpperLimit * TWO_PI, height, 0);
    color c = img.get(int(x), int(y));

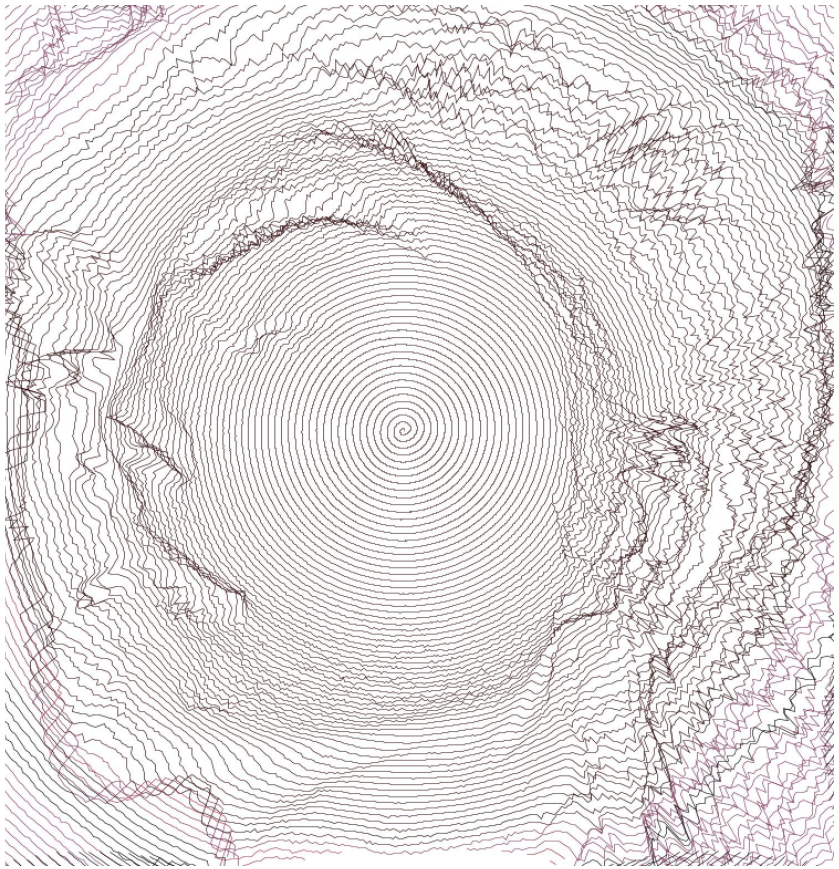

    float rVal = red(c) * sin(frameCount * 0.01);
    float gVal = green(c) * cos(frameCount * 0.01);
    float bVal = blue(c) * sin(frameCount * 0.02);

    period = map(rVal + gVal + bVal, 0, 255 * 3, adj2, 0);

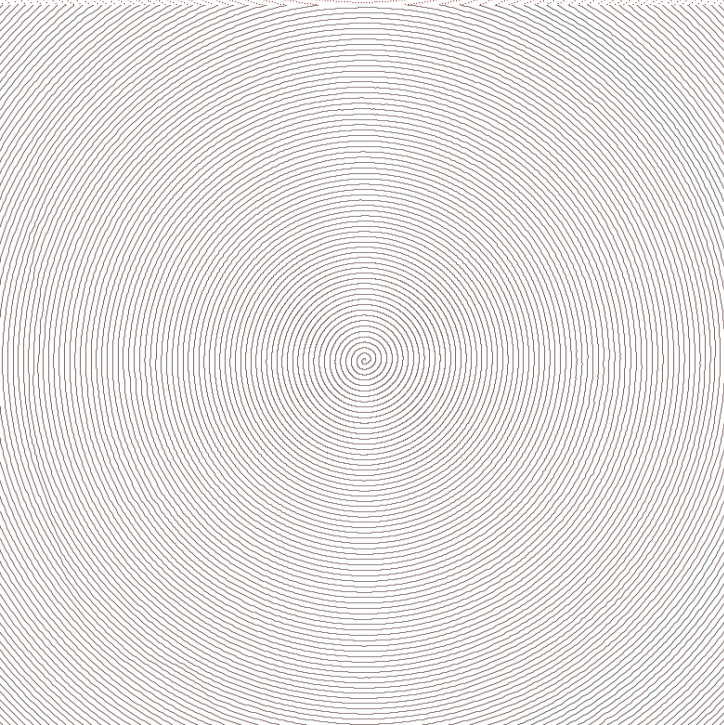

    r = t + pow(t, adj1) * sin(period * t) + 0.5 * sin(2 * t) + 0.2 * cos(3 * t);

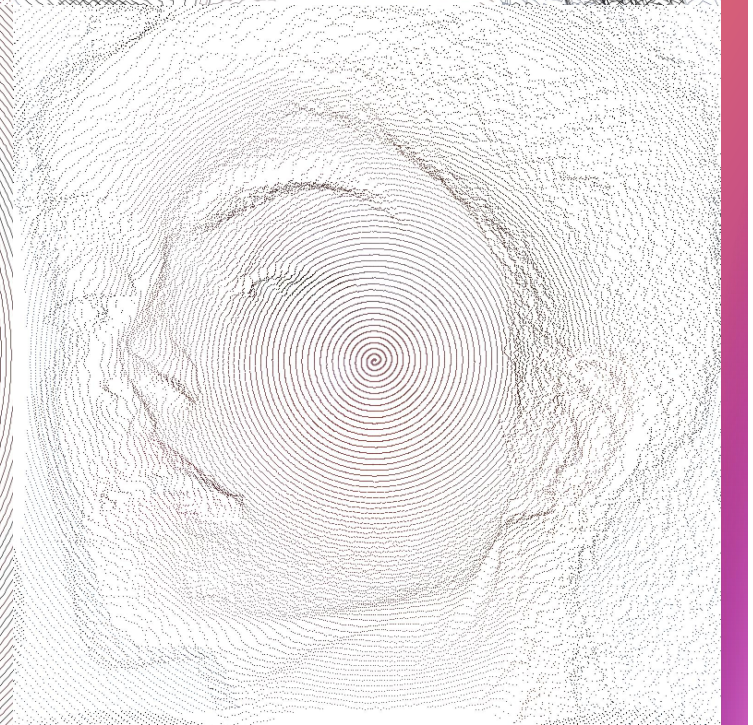
    x = map(r * cos(t), -1 * adjustedUpperLimit * TWO_PI, adjustedUpperLimit * TWO_PI, 0, width);
    y = map(r * sin(t), -1 * adjustedUpperLimit * TWO_PI, adjustedUpperLimit * TWO_PI, height, 0);

    if (x < 0 || x > width || y < 0 || y > height) {
      endShape(OPEN);
      beginShape();
    } else {
      if (form) {
        strokeWeight(1.2);
        stroke(rVal, gVal, bVal);
        point(x, y);
      } else {
        strokeWeight(0.6);
        stroke(rVal, gVal, bVal);
        vertex(x, y);
      }
    }
```









OUTCOME

Version1： circles and lines

by a click

version2： circles and points